

2026 新北市 AI 智慧城市黑客松 · 交通局命題 · 項目二 數據及資料運用

模型訓練 從訓練資料集到線上端點

資料預處理交出 1,528 條訓練序列之後 ——
我們怎麼選演算法、怎麼決定訓練集與測試集、
成果怎麼驗收、端點怎麼建起來。

這一份講什麼

資料怎麼落地、怎麼守門、怎麼格點化，**前一份已經講完**。
這一份從「手上已經有一份訓練資料集」開始。



STEP 01

演算法選型

資料是 1,528 條等距的整數計數序列，而調度需要的是機率預測

STEP 02

決定訓練集

標的選可借車輛數，動態特徵只放未來算得出來的，預測長度 3 小時

STEP 03

設定測試集

訓練期之後的 2026-04 整月，多原點滾動評估

STEP 04

驗收成果

wQL 與 RMSE 兩把尺，跟「什麼都不做」比好多少

STEP 05

建立端點

三段部署，回接後端批次推論寫進資料庫

這一份從哪裡接手

手上已經有的 資料預處理的產物

- **1,528 站 × 30 分鐘等距格點** 一站一條時間序列，右端切在訓練期截止
- **target 允許 null** 缺格留空，不用 0 或前值填補
- **一欄日曆特徵 is_holiday** 來自人事行政總處辦公日曆表
- **train / test 兩個 JSON Lines channel** SageMaker DeepAR 直接可讀的格式
- **站別對照 cat_map.csv 與 meta.json** 把模型裡的編號對回站點

問題框架定下的三條 直接決定後面每一個選擇

- **模型只輸出車數**。燈號、補取車台數、要不要派車，全部由後端規則層算 —— 門檻是使用者可以調的營運參數，調門檻不必重訓模型
- **要支撐三個決定**。先去哪一站、派幾台、要不要派
- **需要的是「+3 小時會怎樣」**。調度有前置時間，看即時資料出發，到場時尖峰已經過了

為什麼把風險判定留在規則層。把「會不會缺車」直接當標籤訓成分類器，每調一次門檻就要重跑整條訓練流程；而且分類器給不出「該補幾台」。

先看資料長什麼樣

資料特性 預處理交出來的東西

- **1,528 條同頻率、等距的序列** 30 分鐘一格，時間軸完全對齊
- **目標值是非負整數計數** 車輛數，最小 0，有上限（車柱數）
- **站與站高度相關** 全市共享同一組通勤節律與週期，捷運站早上空、住宅區早上滿
- **小站資料稀疏** 一天只動幾台，單站看不出節律
- **缺格是常態** 站台斷訊、連缺 3 小時以上就留 null

因此演算法必須 選型的五個條件

- **吃多條序列聯合訓練** 讓資料稀疏的小站借用大站學到的模式
- **支援計數型分佈** 輸出要落在非負整數上
- **輸出機率分佈** 調度是有成本的行動，要知道最壞會多壞；單點預測撐不起「派不派車」
- **容忍 null** 不要求先把缺格補滿
- **一顆模型覆蓋全市** 維運成本可控

五個條件裡最硬的是第三個。缺車與滿站的代價不對稱，而且不對稱的方向依站而異——捷運站早上缺車的代價，遠高於同一站晚上滿位。單點預測把這個不對稱抹平了。

選 SageMaker 內建的 DeepAR

四個理由 對上前一頁的五個條件

- 1 **跨站聯合訓練**。全市 1,528 站共享一組季節性與通勤節律， 稀疏小站借用大站的模式；一顆模型覆蓋全市。
- 2 **原生輸出分位數**。推論直接回 q19／q50／q90 三條， 外面不必再套一層分佈估計。
- 3 **原生吃站別與日曆**。feat_static_cat 放站別、dynamic_feat 放是否假日，兩者都是規格內的欄位。
- 4 **內建容器**。不必自建訓練映像；test:mean_wQuantileLoss 由 SageMaker 直接回報， 評估端少一套要對齊的程式。

沒有走的三條路 以及為什麼

替代方案	放棄的理由
每站一個模型	1,528 個模型的維運成本不可控；小站資料稀疏，單獨訓練學不出通勤節律
訓成分類器 直接預測會不會缺車	門檻 $T = \max(2, 15\% \times \text{車柱})$ 是使用者可調的營運參數， 當成標籤訓進模型，改一次門檻就要重訓；而且分類器給不出「該補幾台」
需求預測 預測借出 / 歸還次數	資料源只有水位快照，借還事件推不回來 —— 同一格內借 3 還 3 看起來完全沒動

先在本地跑，再搬上雲

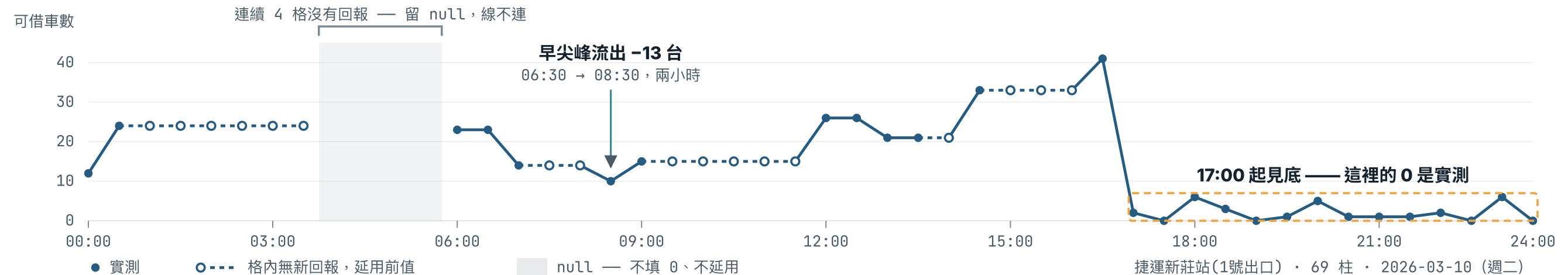
SageMaker 的 DeepAR 基於 GluonTS，**資料格式與超參名稱大致對應**—— 本地調好的東西可以幾乎原樣搬上去。

本地定下的	值	理由
<code>likelihood</code>	<code>negative-binomial</code>	車輛數是非負整數計數；預設的 student-T 支撐在整條實數線上，會吐出負數與小數
<code>embedding_dimension</code>	50	站別嵌入維度。預設 10 對 1,528 個站別太小，塞不下站與站的差異
<code>mini_batch_size</code>	64	讓兩邊組態語意一致，本地量到的東西搬上雲才有參考價值
評估遮罩	只算 <code>is_observed = 1</code> 的格	補值填的格答案等於前一格，混進去算會讓誤差憑空低估約三成

兩邊的分數不可並排。SageMaker 的 test channel 只考每條序列最後 6 格，本地是多切點滾動回測；一邊出 wQL 與 RMSE，一邊出 MAE。兩份實驗帳（`experiments.csv` 與 `sm_experiments.csv`）從頭到尾沒有合併過。

標的：可借車輛數

模型學的是**水位本身** —— 幾點幾分這一站有幾台車，每半小時一格。



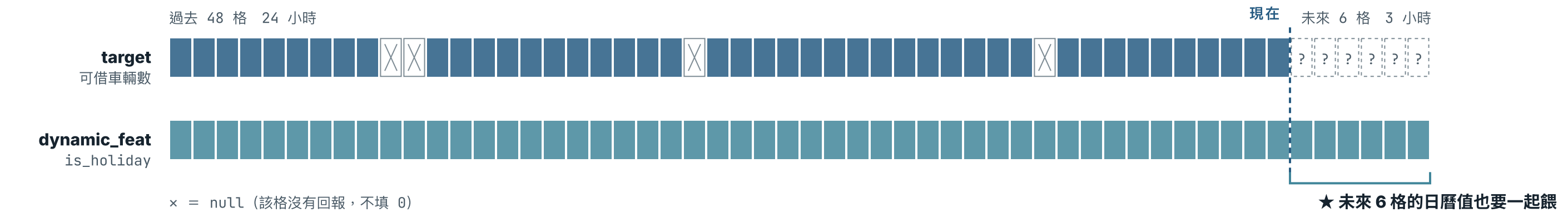
- 1 選 **avail（可借車輛數）** 當標的。它是資料源直接可觀測的量。借還事件推不回來 —— 同一格內借 3 台還 3 台，水位看起來完全沒動。
- 2 缺格留 **null**，不填 **0**。圖上 17:00 之後那一整段的 0 是實測 —— 代表「這站真的一台都沒有」。用 0 補缺格，等於告訴模型有一堆不存在的空站。

- 3 連缺 3 小時以上就不延用前值。圖上 04:00–05:30 那四格因此留空、線不連。短缺口可以補（空心點那些），長斷訊補下去會讓「站台斷線」與「水位真的沒動」在模型眼裡長得一模一樣。

訓練與推論吃同一條管線。訓練用的 `baseline_grid`，與線上推論時餵進去的 48 格歷史，來自同一張表、同一套規則 —— 避免模型在訓練時看到的分佈，和上線後拿到的資料長得不一樣。

特徵：只放未來算得出來的

DeepAR 的輸入規格：動態特徵在推論時**必須連未來 6 格的值一起餵進去**。模型要預測 +30 分鐘到 +180 分鐘，就得先知道那 6 格的特徵值。**所以特徵只能放「現在就算得出未來值」的東西。**



留下的 is_holiday 一欄

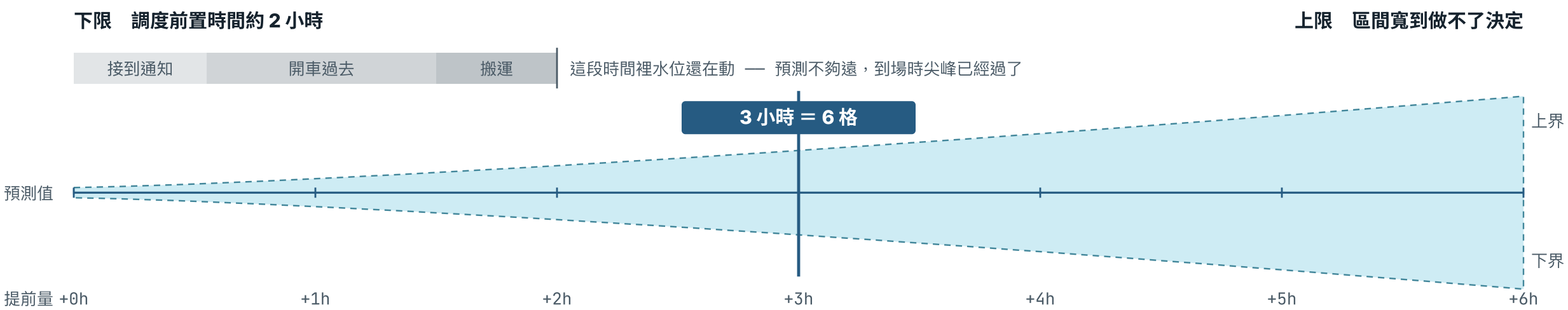
人事行政總處的辦公日曆表已經公告到 2027 年， 任何一天是否放假都查得到 —— 未來 6 格的值隨時算得出來。

出局的 天氣

未來三小時的天氣只能拿**預報**，而預報本身有誤差。 餵進去等於在水位預測上疊加第二層不確定性。

預測長度：3 小時

往前推是被**調度的前置時間**推出去的，往後收是被**預測區間的寬度**收回來的。



下限 至少要 3 小時 前置時間推出來的

調度是有前置時間的行動：接到通知、調度車開過去、把車搬上搬下。這段時間裡水位還在動。

預測只看半小時、一小時，等於看著即時資料出發 —— **到場時尖峰已經過了。**

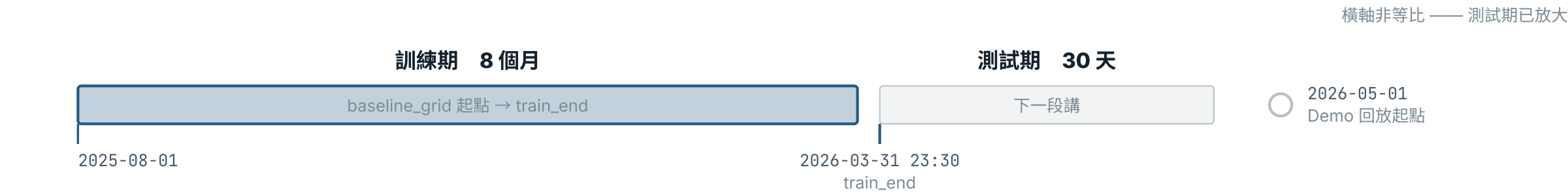
上限 再遠就沒有參考價值 區間寬度收回來的

模型吐的是一個區間。**提前量拉長，區間跟著張開** —— 拉到半天、一天，區間會寬到「可能缺車、也可能滿站」，調度人員看了做不了任何決定。

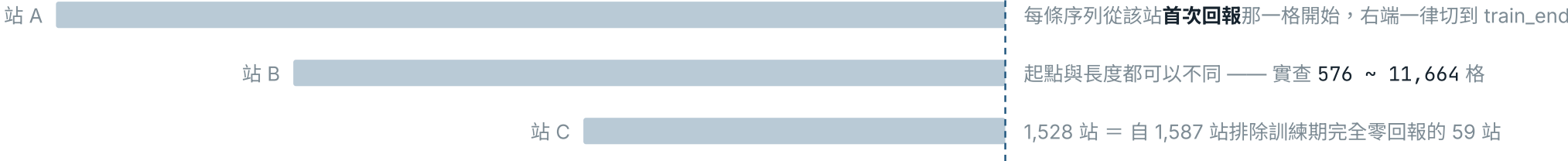
預測的價值在於**它夠窄到能支撐一個動作。**

量過的代價。把預測長度從 2 小時拉到 3 小時，wQL 與 RMSE 各惡化約 9.4%。⚠ 那兩次用的測試集不同，這是約略的量級，不是嚴格的對照實驗。

訓練期與站集合



TRAIN CHANNEL 1,528 行 = 1,528 站 × 1 條



訓練期止於 **3/31**，是為了 **Demo 的誠實性**。比賽 Demo 的虛擬時鐘從 2026-05-01 起跑。先前的模型訓練期一路到 6-18，拿它 Demo 五月，等於用一顆**見過答案**的模型。所以重訓，把訓練期截止搬回 3-31 —— 代價是少了 2.5 個月的資料。

站數一變，站別編號整批重排。編號由站點代碼升冪決定，所以 cat_map.csv 與後端的站別欄位必須同步換。沒換的話每個站會拿到**別人的**預測，而且模型照樣輸出合理數字，沒有任何錯誤訊息。

站 A／B／C 是示意。防線是 meta.json 的 names_sha 指紋，換模型時比對兩邊。

超參怎麼定下來的

定版十一格 youbike-deepar-d2604v2-r2

prediction_length	6	+30m ~ +180m
context_length	48	過去 24 小時
time_freq	30min	
epochs	80	
learning_rate	5e-4	
mini_batch_size	64	
num_cells	40	
num_layers	2	
dropout_rate	0.1	
likelihood	negative-binomial	
embedding_dimension	50	
num_dynamic_feat	1	is_holiday
cardinality	auto	實際 1,528

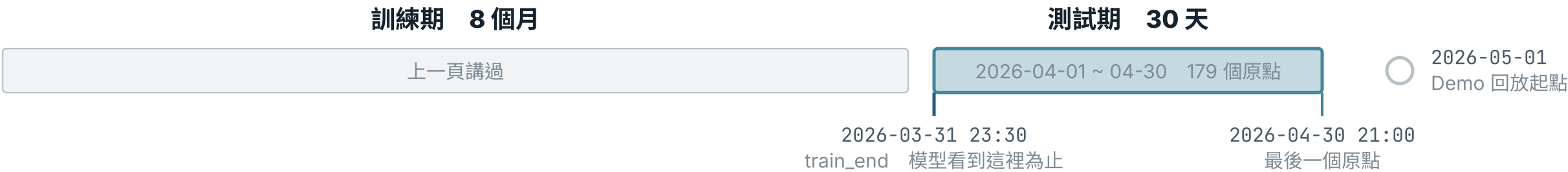
38	8.17 hr	約 \$4
training job 35 完成 / 3 失敗	instance-hour ml.c5.2xlarge	全部訓練成本

每一格的出處與強度 節錄

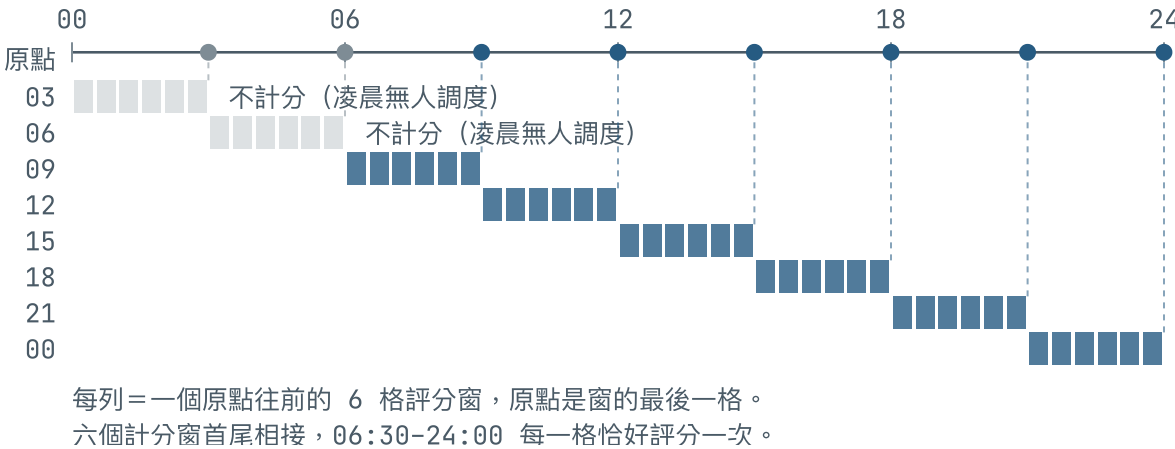
超參	出處	強度
context_length 48	加長到 96 格，兩種預測長度下都測不出差異；48 較便宜	量測（兩次陰性）
learning_rate 5e-4	1e-3 跑到 200 epochs 會 NaN；降下來買到 80 epochs 的餘裕	量測（穩定性）
num_cells 40	加到 80 試了三次都沒有效果，還多付 14~40% 的時間	量測（三次陰性）
epochs 80	來自單點排名。真正站得住的只有「10 → 40 有效」這一段	▲ 排名
num_layers 2 dropout_rate 0.1	沿用預設與本地組態	▲ 未測
其餘六格	資料型態與架構決定的（計數型分佈、站別數、未來值可知的特徵）	設計決定

誠實地說：十一格裡三格靠量測、一格靠排名、兩格沒測過，其餘本來就沒有調參空間。這條路上最貴的產物是一條紀律——沒量過雜訊帶的單次結果不能解讀。

測試集怎麼切



- 1 按時間切，不隨機切。測試集整段取在訓練期之後 —— 2026-04 一整個月，模型訓練時一格都沒看過。隨機抽格會讓模型從前後文猜出答案。
- 2 多原點滾動。每 3 小時放一個預測原點，每個原點往後預測 6 格。一個月鋪滿 179 個原點，每一站都被反覆考到不同的時段、不同的星期。
- 3 評分窗首尾相接、不重疊。六個窗接起來蓋滿 06:30 到 24:00，這段時間的每一格恰好被評分一次，沒有哪一格被重複計分。



評分窗與它的邊界

原點	03	06	09	12	15	18	21	00
評分窗	00:30-03:00	03:30-06:00	06:30-09:00	09:30-12:00	12:30-15:00	15:30-18:00	18:30-21:00	21:30-00:00
是否評分	x	x						
佔評分格	—	—	16.7%	16.6%	16.7%	17.0%	17.0%	16.0%

為什麼排掉凌晨兩個窗 00:30-06:00

那段時間沒有人調度。留著它，等於把兩成的評分格花在營運上用不到的時段，一路稀釋掉模型在營運時段的表現。
考題要對著決策出。

排掉之後 兩個附帶的好處

凌晨站台整夜無回報、缺格率特別高 —— 排掉它連帶把整體**評分率拉到 96.3%**。
而且剩下的六個窗，正好把實測的早尖峰 06:30-08:30 **整段**包進同一個窗裡。

179

預測原點
2026-04 每 3 小時一個

1,528

站數
與訓練集同一組

273,154

測試序列條數

1,578,169

實際評分格 (96.3%)
缺格不計分

代價要一起講。凌晨 00:30-06:00 不在測試集裡，所以**那段時間的預測品質完全沒有量測過**。若之後有夜間調度需求，本份的成績不能沿用。

兩把尺

wQL 加權分位數損失 主要指標

$$wQL(q) = 2 \cdot \sum QL_q(y, \hat{y}_q) / \sum |y|$$

$$QL_q = q \cdot \max(y - \hat{y}_q, 0) + (1 - q) \cdot \max(\hat{y}_q - y, 0)$$

- **量的是整條預測分佈**。它同時罰「下界壓得不夠低」與「上界抬得不夠高」，而不只看中位數對不對。
- **正規化過**。分母是實際車數的總和，所以 30 柱大站與 8 柱小站的分數可以相加 —— 未正規化的誤差會整個被大站主導。
- **數字愈小愈好**，0 是完美。

口徑要一起記。本專案把 test_quantiles 覆寫成 [0.1, 0.5, 0.9]，所以後面的 wQL 是三個分位數的平均。DeepAR 預設是九個，用九分位算出來的值會系統性偏高約 20% —— 任何要跟本份數字並排的成績，都必須用同一組分位數。

RMSE 均方根誤差 併記指標

$$RMSE = \sqrt{\text{mean}((y - \hat{y}_{q50})^2)}$$

以中位數 q50 當點預測

- **白話就是「平均差幾台車」**，單位與車數相同，直接看得懂。
- **對大誤差特別敏感**。平方項讓「差 10 台」那種錯誤被放大 —— 而那正是調度最怕的。wQL 對離群點相對寬容，兩把尺併用才蓋得住。
- **兩把尺在站點規模上排序相反**（P18），所以兩個都留著。

先定義「什麼都不做」

要證明訓一顆模型有正當性，得先知道**不訓模型能拿幾分**。調度人員完全可以假設「接下來三小時車數不變」來排班——那不需要任何模型、不花任何錢。**模型要證明的，就是它比這個假設好。**

對照組	做法	量的是什麼
點預測 persistence	預測值 = 最後一個觀測值，三條分位數同值	「什麼都不做」的底線
機率型 persistence ★ 正式合格線	最後一個觀測值 + 訓練期殘差的第 q 分位，截斷在 0 以上	對 wQL 公平的底線——同樣有分佈的對手
日回歸	昨天同一時刻的值（往前 48 格）	單靠「日週期」值多少分
週回歸	上週同一時刻的值（往前 336 格）	單靠「週週期」值多少分

為什麼一定要有「**機率型 persistence**」。wQL 量的是分佈。點預測把機率質量全押在一個值上，會被結構性地多罰一截——拿模型去比點預測，有一部分優勢來自「有分佈 vs 沒分佈」，那並不是預測能力。機率型 persistence 把這件事補平：從訓練資料收集殘差，逐個提前量取分位數，讓對手也有一條合理的區間。

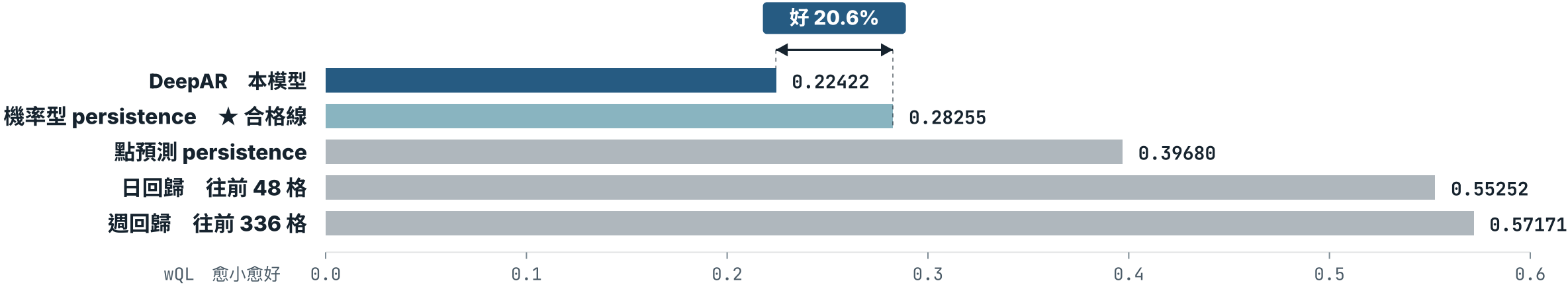
訓練期殘差的分位數（台）

	q10	q50	q90
+30m	-2	+0	+2
+60m	-4	+0	+4
+90m	-5	+0	+5
+120m	-6	+0	+6
+150m	-7	+0	+7
+180m	-8	+0	+8

半小時內，八成的站動不到 2 台

成果：贏「什麼都不做」兩成

方法	wQL	RMSE	模型的改善
DeepAR 本模型	0.22422	5.0417	—
機率型 persistence ★ 合格線	0.28255	6.5698	wQL -20.6% RMSE -23.3%
點預測 persistence	0.39680	6.5698	-43.5%
日回歸 往前 48 格	0.55252	8.0097	-59.4%
週回歸 往前 336 格	0.57171	8.2323	-60.8%



wQL 降 20.6%，不等於營運改善 20.6%。兩者之間的換算需要一套調度成本模型（一趟出車的成本、一次借不到車的代價），本專案沒有做。這是一個誠實的量測，不是達標宣告。

報平均，不報冠軍

單次訓練的分數裡有**隨機種子的運氣**。同一組態送三個 job，才知道這個組態值多少分。

job		test wQL	test RMSE	計費秒
d2604v2-r1		0.22153	4.9881	1,671
d2604v2-r2	★ 部署這顆	0.22422	5.0417	1,641
d2604v2-r3		0.22567	5.1004	1,646
平均		0.22381	5.0434	4,958
全距		0.00414	0.1123	

三顆分開看，對合格線的改善是 **21.6% / 20.6% / 20.1%** —— 最差的一輪也在兩成以上。這個結論不靠平均值撐起來，也不靠單次運氣。

為什麼部署 r2 而不是分數最好的 r1

三顆在統計上等價。r2 的 0.22422 最接近三輪平均 0.22381 —— 部署的那一顆，自己的成績與「這個組態值多少分」一致，對外說明時少一層轉折。
挑 r1 等於把隨機種子的運氣算進模型的成績裡。

三次的全距不能當成「雜訊變窄了」。n = 3 的全距會系統性低估真實的離散度。站得住的說法只有一句：**雜訊沒有變大**。

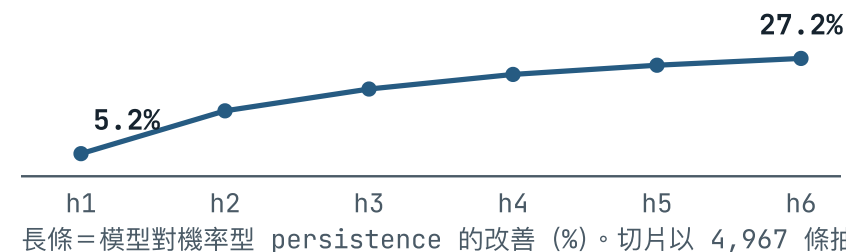
DeepAR 不支援接續訓練。每個 job 都從隨機初始化重訓，上一顆模型的權重一個 byte 都不會被讀進來 —— 所以這是三顆獨立的模型，而「重跑一次 job 來評新測試集」會換掉一顆模型。

贏在哪裡

聚合成一個數字會把難與易混在一起。拆開來看，**模型的價值集中在難的地方**。

① 依提前量

每一格都贏，優勢單調上升



依提前量 +30m → +180m

每一格都贏，而且優勢單調上升：

+30 分鐘贏 **5.2%** → +180 分鐘贏

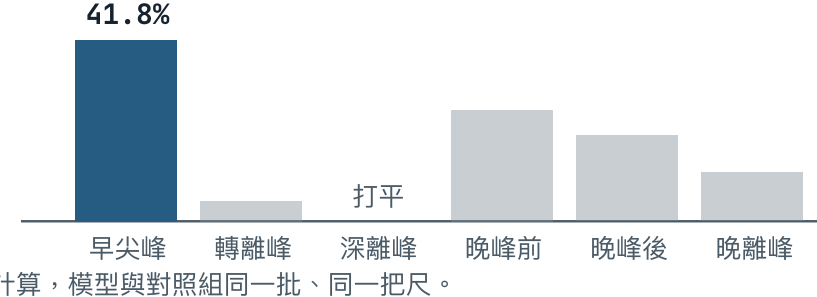
27.2%

半小時後「跟現在一樣」是很強的猜測，但撐不到三小時。

而調度需要的正是 **2~3 小時的前置量**。

② 依時段

價值集中在最難的那一段



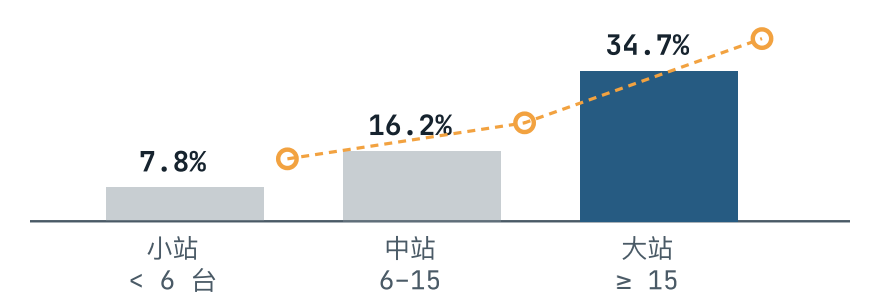
依時段 六個評分窗

早尖峰 06:30–09:00 贏 **41.8%** —— 而那同時是**全天最難**的一段（對照組 wQL 0.4049，是日間離峰的 1.83 倍）。

日間深離峰打平（-0.1%）：**車不動的時段不需要模型**。

③ 依站點規模

相對誤差與絕對誤差排序相反



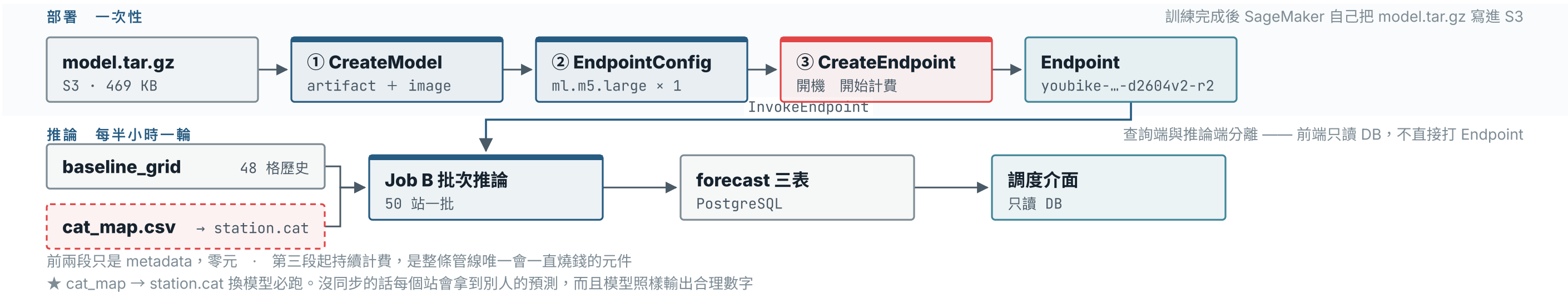
依站點規模 尖峰可借車數

大站 ≥ 15 台贏 **34.7%**、中站 16.2%、小站 7.8%。

大站正是尖峰打得最重、絕對誤差最大的一群（RMSE 10.53 對小站 3.60）。

尖峰時段是實測出來的。掃 2,650 萬列的流動強度（每站每 30 分鐘的車數變動量），量到早尖峰落在 **06:30–08:30**（峰值 07:00）、晚尖峰落在 **17:00–19:30**（峰值 18:30）——與大眾運輸的慣例值差半小時到一小時。原點按整點排，差這一格就會把尖峰切在窗中間。

從模型檔到線上端點



1

CreateModel

綁定模型檔與演算法映像。訓練完成後 SageMaker 自己把 model.tar.gz 寫進 S3（469 KB），我們只給輸出 bucket，要用時再問回來。

2

CreateEndpointConfig

指定機型與台數：ml.m5.large × 1。這一段與上一段都只是 metadata，零元。

3

CreateEndpoint

開機。**從這裡開始計費** —— endpoint 是整條管線唯一持續計費的元件，其餘都是一次性成本。

回接後端三件事。 ① 站別對照 cat_map.csv 灌進後端站別欄位（換模型必跑） ② 推論用的 48 格歷史也取自同一張 baseline_grid ③ 批次推論 50 站一批呼叫端點，結果寫進資料庫。

查詢端與推論端分離。 前端只讀資料庫，不直接打端點 —— 使用者的查詢不受推論延遲影響，端點也不會被前端流量放大。

上線之後

兩件已經量到的事 2026-09-10 校準

- 1 線上只餵 48 格歷史，吃掉約一半效益。 同一批序列只改歷史長度：完整歷史贏合格線 21.4%、一週（336 格）19.4%、**48 格只剩 11.7%**。336 格幾乎全部救回 —— 那正好對應演算法對 30 分鐘頻率 最遠的那組回看點。△ 已量測，尚未修改；改了要重新校準下界。
- 2 下界分位數重新校準，**0.19 → 0.18**。實測覆蓋率 88.8%（目標 90%），下界壓得不夠低，缺車會被漏掉。校準後回到 89.4%。△ 早尖峰仍只蓋 86.0% — 單一個數字修不到它，要依時段分層（逐時段全距 0.14，逐提前量只有 0.07）。

合格的是模型，打折的是部署組態。這個區分很重要 —— 修法是改一個常數，不必重訓。

模型的使用邊界 這五件事本模型做不到

- 超過 3 小時的預測 預測長度寫死為 6 格，更長的提前量完全沒有量測過
- 模擬調度介入後的水位 模型預測的是**無介入**情境。調度車一到，實際水位就偏離預測
- 凌晨 00:30-06:00 的準確度宣告 那兩個窗刻意排出測試集，成績裡沒有那段的任何一格
- 訓練期外太遠的時間 訓練期止於 2026-03-31，Demo 回放五月是刻意設計；更晚的時期會踩到分佈漂移
- 車柱數變動後的滿站判定 模型只預測可借車數，車柱數取自站點屬性；擴柱後滿站門檻會失準

執行期的自我揭露。每次推論回應都附帶警語，並回報這一筆用了幾格補值、幾格延用前值、幾格仍是空的 —— 呼叫端可以自己判斷這筆預測建立在多少實測資料上。